

BioAttend v2 — System Report

System: Smart Biometric Attendance System (Android + ESP32 REST + R307S)

Institution: Marwadi University — ICT Department

Timezone: Indian Standard Time (IST - Asia/Kolkata)



BioAttend v2 — Complete System Documentation & Project Report

Project Title: BioAttend v2 — Smart Biometric Attendance System

Target Organization: Marwadi University — ICT Department

Timezone: Indian Standard Time (IST - Asia/Kolkata)

Technology Stack: Android (Kotlin, Room DB, Retrofit, Coroutines), ESP32 (REST API, SSD1306 OLED, R307S Fingerprint Sensor), Web Simulator (HTML5, CSS3, JavaScript)



Table of Contents

- [1. System Overview & Architecture](#)
- [2. Key System Components](#)
- [3. Complete Technical & Non-Technical Feature List](#)
 - [◦ Authentication & Gmail OTP Verification](#)
 - [◦ Top-Left Side Navigation Drawer & Profile Header](#)
 - [◦ Dedicated Enrolled Sensor IDs List](#)
 - [◦ R307S Biometric Enrollment & Hardware Polling Loop](#)
 - [◦ User Data & Excel Schema Management](#)
 - [◦ Indian Standard Time \(IST\) Real-Time Timetable](#)
 - [◦ Dynamic UI Accessibility & Zoom Controls](#)
 - [◦ Interactive Web & Hardware Simulator](#)
- [4. Hardware REST API Specifications](#)
- [5. Summary of Critical Bug Fixes](#)
- [6. Build System & Verification Matrix](#)



System Overview & Architecture

BioAttend v2 is an end-to-end smart attendance system designed to replace traditional paper registers with real-time biometric verification using optical fingerprint sensors, local database persistence, and REST API

communication with an ESP32 microcontroller.

```
flowchart TD
    subgraph Android_Application_Kotlin [Android Application (Kotlin)]
        UI[Main Dashboard / Timetable]
        NAV[Top-Left Avatar Drawer]
        AUTH[Gmail OTP Verification]
        DB[(Room SQLite Local Database)]
        RETRO[Retrofit REST Client]
    end

    subgraph ESP32_Microcontroller [ESP32 Microcontroller]
        API[ESP32 Async Web Server]
        R307S[R307S Optical Fingerprint Sensor]
        OLED[128x64 SSD1306 OLED Display]
    end

    subgraph Web_Hardware_Simulator [Web & Hardware Simulator]
        WEB[web_simulator/index.html]
    end

    AUTH -->|User Session| UI
    NAV -->|Enrolled Sensor IDs| DB
    UI -->|Query Today's IST Schedule| DB
    RETRO -->|POST /api/fingerprint/enroll| API
    RETRO -->|Poll GET /api/fingerprint/enroll/status| API
    RETRO -->|POST /api/oled/show| API
    API -->|Read / Store Templates| R307S
    API -->|Render Messages & Status| OLED
    WEB -.->|Mirrors Android UI & Hardware| API
```

Key System Components

1. Android App (`com.mpc.bioattend`):

- Built in Kotlin targeting Android 14 (API 34).
- Manages student records, timetable schedules, attendance logs, and Wi-Fi hardware configuration.
- Uses Android Jetpack Room for persistent local storage and Retrofit2 for network communications.

2. ESP32 Microcontroller & Biometric Hardware:

- Acts as an HTTP REST server listening on local IP (e.g., `http://192.168.1.25`).
- R307S Optical Fingerprint Sensor stores up to 512 biometric template slots.
- 0.96" SSD1306 OLED Screen displays live instructions (`Enter Your ID` , `Place Finger` , `Enrolled Success`).

3. Web Simulator (`web_simulator/index.html`):

- Single-page responsive HTML/CSS/JS application matching the mobile app layout and physical hardware.
 - Simulates real-time OLED rendering, sensor touching, drawer navigation, and REST API logging.
-

Complete Technical & Non-Technical Feature List

1. Authentication & Gmail OTP Verification

- **User Experience (Non-Technical):** Users sign in or register with their full name and Gmail address. A verification code is sent directly to their email inbox. Fake or hardcoded default codes have been completely removed.
 - **Technical Details:**
 - Validates email strings to end with `@gmail.com`.
 - Dispatches OTP emails via JavaMail SMTP in background coroutines.
 - Automatically extracts user initials (e.g. "GN" for Mitesh Solanki) and updates both the main dashboard header avatar and side drawer profile card.
-

2. Top-Left Side Navigation Drawer & Profile Header

- **User Experience (Non-Technical):** Tapping the glowing avatar icon in the top-left corner slides out a navigation drawer displaying user profile details at the bottom-left and quick links to all application screens.
 - **Technical Details:**
 - Drawer menu options:
 -  **Connectivity & Wi-Fi**
 -  **User Data (Excel Schema)**
 -  **Time Table Schedule**
 -  **Enrollment Manager**
 -  **Enrolled Sensor IDs**
-

3. Dedicated Enrolled Sensor IDs List

- **User Experience (Non-Technical):** The Enrolled Sensor IDs button is located directly in the side navigation drawer. Tapping it pops up a clean dialog showing hardware memory usage and a detailed list of every mapped student without any repeated or duplicate entries.
- **Technical Details:**
 - Formatted in `MainActivity.kt` and `EnrollmentManagerActivity.kt` to render all 8 student data columns: `Slot ID`, `Sr.No`, `Student ID`, `Name`, `Class/Sem`, `GR.No`, `Enrollment No`, `Attended Classes`.
 - Employs `.distinctBy { it.fingerprintId }` and `.distinct()` filtering to prevent duplicate database records from populating multiple lines.

4. R307S Biometric Enrollment & Hardware Polling Loop

- **User Experience (Non-Technical):** Typing an ID in the enrollment section instantly updates the physical OLED display to show ID: X . Tapping **Start Enrollment** prompts the user to touch the sensor twice. A completion popup appears **only after** the finger scan completes successfully, displaying the exact enrolled ID.
 - **Technical Details:**
 - **Nested JSON Parsing:** Resolved ESP32 JSON payloads (`{"success":true,"data":{"fingerprint_id":1,"finished":false}}`) by adding `FingerprintDataDetails` and the `effectiveFingerprintId` getter in `FingerprintResponse.kt` .
 - **Coroutine Polling:** `executeEsp32Enrollment()` issues `POST /api/fingerprint/enroll` and repeatedly polls `GET /api/fingerprint/enroll/status` every 1,000ms until `finished == true` .
 - Holds back dialog popups during hardware capture to prevent premature alerts with ID 0 .
-

5. User Data & Excel Schema Management

- **User Experience (Non-Technical):** Displays student records in an easy-to-read table. Example/dummy data has been removed so only actual imported or created students are saved.
 - **Technical Details:**
 - Schema mapping: `Sr.No` , `Student ID` , `Name` , `Class/Sem` , `GR.No` , `Enrollment No` , `Fingerprint ID` , `Attended Classes` .
 - Supports CSV import/export, student searching, editing, and deletion.
-

6. Indian Standard Time (IST) Real-Time Timetable

- **User Experience (Non-Technical):** Manages daily lecture schedules synchronized strictly with Indian Standard Time (IST - Asia/Kolkata / Mumbai). Displays live status badges for current classes.
 - **Technical Details:**
 - `TimetableActivity.kt` queries `TimetableEntity` records sorted by IST start/end timestamps.
 - Dynamically renders status badges:  **ACTIVE NOW** ,  **UPCOMING** , or  **COMPLETED** .
-

7. Dynamic UI Accessibility & Zoom Controls

- **User Experience (Non-Technical):** High-contrast **Zoom In (+)**, **Zoom Out (-)**, and **100% Reset** controls allow users to scale table text and schedule cards for easy reading.
 - **Technical Details:**
 - Implemented in `activity_user_data.xml` , `activity_timetable.xml` , and `web_simulator/index.html` .
 - Dynamically resizes container layout view matrices between `70%` and `200%` .
-

8. Interactive Web & Hardware Simulator

- **User Experience (Non-Technical):** A web-based interactive preview allowing testing of the phone interface, side drawer, OLED screen, finger touches, and log output directly from any web browser.
 - **Technical Details:**
 - Located at `web_simulator/index.html` .
 - Features real-time Javascript event handlers for enrollment status polling, OLED state transitions, and responsive drawer toggling.
-

Hardware REST API Specifications

Method	Endpoint	Description	Request Payload / Params	Response Structure
GET	/api/health	Hardware health check	None	<pre>{"success": true, "status": "ONLINE"}</pre>
POST	/api/oled/show	Update OLED text display	<pre>{"title": "Enter Your ID", "message": "ID: 1"}</pre>	<pre>{"success": true}</pre>
POST	/api/fingerprint/enroll	Start fingerprint enrollment	<pre>{"fingerprint_id": 1}</pre>	<pre>{"success": true, "data": {"fingerprint_id": 1, "finished": false}}</pre>
GET	/api/fingerprint/enroll/status	Poll enrollment progress	None	<pre>{"success": true, "data": {"fingerprint_id": 1, "finished": true, "state": "SUCCESS"}}</pre>
GET	/api/fingerprint/list	List used sensor slot IDs	None	<pre>{"success": true, "data": {"used_ids": [1, 2], "used_count": 2, "available_count": 510}}</pre>
DELETE	/api/fingerprint/{id}	Delete fingerprint template	Path parameter {id}	<pre>{"success": true, "message": "ID deleted"}</pre>

Summary of Critical Bug Fixes

[!!IMPORTANT] Issue 1: Premature Popup Alert Showing Fingerprint ID 0

- *Root Cause:* App assumed initial HTTP 202 response meant hardware capture was complete, and failed to parse `fingerprint_id` nested inside the `data` block.
- *Resolution:* Implemented status polling on `/api/fingerprint/enroll/status` until `finished == true` and added `effectiveFingerprintId` accessor in `FingerprintResponse.kt`.

[!!IMPORTANT] Issue 2: Incomplete & Duplicate Enrolled Sensor IDs List

- *Root Cause:* Enrolled list dialog only displayed student names and doubled entries when multiple database rows referenced the same ID.
- *Resolution:* Enforced `.distinctBy { it.fingerprintId }` and formatted text output to include all 8 User Data columns (Slot ID , Sr.No , Student ID , Name , Class/Sem , GR.No , Enrollment No , Attended Classes).

[!!IMPORTANT] Issue 3: Enrolled List Button Navigation

- *Root Cause:* The Enrolled List button was nested inside the Enrollment Manager view.
- *Resolution:* Created a dedicated  Enrolled Sensor IDs button directly in the top-left Side Navigation Drawer.

Build System & Verification Matrix

- **Android Gradle Plugin (AGP):** `8.2.2`
- **JDK Version:** Temurin JDK 17 (`C:\Users\lavan\.jdk\temurin-17.0.20.1`)
- **Compilation Command:** `app\gradlew.bat compileDebugSources`

Module	Verification Test	Status
Fingerprint Model	<code>FingerprintResponse.kt</code> nested <code>dataDetails</code> parsing	✓ PASSED
Enrollment Polling	<code>EnrollStudentActivity.kt</code> 1-second coroutine loop	✓ PASSED
Enrolled List Formatting	<code>MainActivity.kt</code> 8-column display & deduplication	✓ PASSED
Web Simulator	<code>web_simulator/index.html</code> live preview sync	✓ PASSED
Gradle Compilation	Complete debug build compilation	✓ BUILD SUCCESSFUL in 3m 55s